DeepSeek V4.1 Flash 模型结构，以及方案 B 在本机的布局

40 层 · 隐藏维度 5,120 · 4 路超连接残差流 · 词表 129,280 · 上下文 1M（YaRN 16 倍，原生 64K）· 总参数 763B

逐层结构

左边两条色条：这一层的稠密部分 / 路由专家放在哪里：GPU0 GPU1 主机内存

Token 嵌入 129,280 × 5,120（BF16）→ 复制成 4 路残差流，之后每层都在 4 路上读写

编码器 第 0–19 层

提示词的每个 token 都算；每层 = 注意力 + MoE（384 个专家选 6 个，外加 1 个共享专家）

L0	滑动窗口注意力：最近 128 个 token	
L1	滑动窗口注意力：最近 128 个 token	Engram
L2	压缩比 2 · 写压缩 KV 和索引键 · 自己选 top-512	KV 源 索引源
L3	压缩比 2 · 读 L2 的压缩 KV 和 L2 的 top-512	
L4	压缩比 2 · 读 L2 的压缩 KV 和 L2 的 top-512	
L5	压缩比 2 · 读 L2 的压缩 KV 和 L2 的 top-512	
L6	压缩比 2 · 读 L2 的压缩 KV 和 L2 的 top-512	
L7	压缩比 2 · 读 L2 的压缩 KV 和 L2 的 top-512	
L8	压缩比 2 · 写压缩 KV 和索引键 · 自己选 top-512	KV 源 索引源
L9	压缩比 2 · 读 L8 的压缩 KV 和 L8 的 top-512	
L10	压缩比 2 · 读 L8 的压缩 KV 和 L8 的 top-512	
L11	压缩比 2 · 读 L8 的压缩 KV 和 L8 的 top-512	
L12	压缩比 2 · 读 L8 的压缩 KV 和 L8 的 top-512	
L13	压缩比 2 · 读 L8 的压缩 KV 和 L8 的 top-512	
L14	压缩比 2 · 写压缩 KV 和索引键 · 自己选 top-512	KV 源 索引源 Engram
L15	压缩比 2 · 读 L14 的压缩 KV 和 L14 的 top-512	
L16	压缩比 2 · 读 L14 的压缩 KV 和 L14 的 top-512	
L17	压缩比 2 · 读 L14 的压缩 KV 和 L14 的 top-512	
L18	压缩比 2 · 读 L14 的压缩 KV 和 L14 的 top-512	
L19	压缩比 2 · 读 L14 的压缩 KV 和 L14 的 top-512	

解码器 第 20–39 层

CED：预填充时只算后面还能通过滑动窗口看到的 token，越靠后的层越少，第 39 层只算最后一个

L20	压缩比 1 · 写全局压缩 KV（第 20–39 层共享）· 选 top-512 和候选块	KV 源 索引源 候选块
L21	压缩比 1 · 读 L20 的压缩 KV 和 L20 的 top-512	
L22	压缩比 1 · 读 L20 的压缩 KV 和 L20 的 top-512	
L23	压缩比 1 · 读 L20 的压缩 KV 和 L20 的 top-512	
L24	压缩比 1 · 读 L20 的压缩 KV · 在候选块里重选 top-512	索引源
L25	压缩比 1 · 读 L20 的压缩 KV 和 L24 的 top-512	
L26	压缩比 1 · 读 L20 的压缩 KV 和 L24 的 top-512	
L27	压缩比 1 · 读 L20 的压缩 KV 和 L24 的 top-512	
L28	压缩比 1 · 读 L20 的压缩 KV · 在候选块里重选 top-512	索引源
L29	压缩比 1 · 读 L20 的压缩 KV 和 L28 的 top-512	
L30	压缩比 1 · 读 L20 的压缩 KV 和 L28 的 top-512	
L31	压缩比 1 · 读 L20 的压缩 KV 和 L28 的 top-512	
L32	压缩比 1 · 读 L20 的压缩 KV · 在候选块里重选 top-512	索引源
L33	压缩比 1 · 读 L20 的压缩 KV 和 L32 的 top-512	
L34	压缩比 1 · 读 L20 的压缩 KV 和 L32 的 top-512	
L35	压缩比 1 · 读 L20 的压缩 KV 和 L32 的 top-512	
L36	压缩比 1 · 读 L20 的压缩 KV · 在候选块里重选 top-512	索引源
L37	压缩比 1 · 读 L20 的压缩 KV 和 L36 的 top-512	
L38	压缩比 1 · 读 L20 的压缩 KV 和 L36 的 top-512	
L39	压缩比 1 · 读 L20 的压缩 KV 和 L36 的 top-512	

最终 RMSNorm → 输出头 5,120 × 129,280（BF16）→ 下一个 token 的 logits

DSpark 草稿模型：3 层，每层 128 个专家（MXFP4，7.4 GiB），用于投机解码；mx-llama.cpp 在 V4.1 上暂不支持

单层内部（以解码器层为例）

4 路残差流 · 超连接混合：4 路加权合成 1 路，权重经 Sinkhorn 归一化（20 次迭代）

注意力

RMSNorm 之后；64 个查询头，键值只有 1 个头（K = V，512 维）

查询

x → q_a 5,120→1,280（低秩）→ 范数
→ q_b → 64 头 × 512 维
后 64 维做 RoPE（YaRN）
q_a 的输出也给稀疏索引器

键值

滑动窗口：最近 128 个 token 的原始 KV
压缩 KV：每 r 个 token 合成一行
（r = 2 或 1，在 KV 源层写入）
索引器只放行 top-512 个压缩行

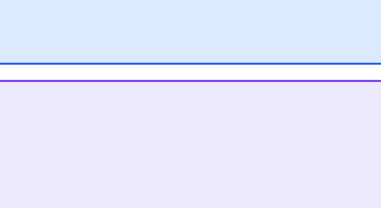
注意力计算

flash attention：128 个窗口行 + 512 个压缩行，每个头一个 sink
输出投影：分 8 组低秩（秩 1,024）→ 5,120，写回 4 路残差流

MoE 前馈

超连接再混合出输入 → RMSNorm

路由：给 384 个专家打分，选 6 个，权重归一化后 × 1.5
路由专家：SwiGLU（clamp 10），5,120 → 2,304 → 5,120
MXFP4，每个专家约 18.8 MB，每层共 7.2 GB
共享专家 1 个：每个 token 都算
输出加权求和后写回 4 路残差流



384 个专家里选 6 个

Engram 查表

只在第 1、14 层：按最近 2–4 个 token 的 n-gram 哈希查两张大表
每张表约 3.84 亿行 × 256 维，每 token 读 24 行（3 种长度 × 8 头），门控后加回残差流

稀疏注意力索引器（第 2/8/14/20/24/28/32/36 层）

索引查询

q_a 输出 → wq_b
→ 32 头 × 128 维
→ RoPE → FP4

索引键

压缩器 latent → wk
→ 范数 → RoPE → FP4
KV 源层写入缓存

头权重

proj(x) / √(128·32)
每个头一个权重

分数 = Σ_h ReLU(q_h · k) · w_h，按查询可见的位置选 top-512 个压缩行
第 20 层：每 8 行一块，按块内最高分挑 2,048 块（最新的块固定保留）
第 24/28/32/36 层只在这些块里重选；其余层沿用最近索引源层的 top-512

方案 B 的存储布局（本机）

GPU0 · RTX PRO 6000 Max-Q，95.6 GiB
第 0–11 层整层：专家 80.6 GiB + 稠密部分

GPU1 · RTX PRO 6000 Max-Q，95.6 GiB
第 12–39 层的稠密部分，第 12–21 层的专家 67.2 GiB
专家缓存 14 GiB：第 22–39 层的热专家，约 780 个槽位

主机内存 · 192 GiB DDR5（锁页）
第 22–39 层的专家 121 GiB
解码时缓存未命中的专家：一半上传进缓存，一半由 CPU 直接算

NVMe · Samsung 9100 PRO
两张 Engram 表约 97 GiB（MXFP4），经页缓存读取，预填充前整批预取

图例 KV 源 写压缩 KV 索引源 选 top-512 候选块 Engram

来源：GGUF 元数据（mxxm-t/DeepSeek-V4.1-Flash-GGUF）、官方 inference/model.py；本机布局和实测见 CLAUDE.md 与 docs/scheduler.md（2026-10-25）